Return values and local state

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

Before you start

A function show_remaining(capacity, used) prints capacity - used. What does calling it with (9, 4) display? Does defining it alone display that result?

Plan time to read, investigate and explain. Keep predictions separate from observations.

STUDENT LESSON · 1 OF 18

Your mission

> Start with the target: Use a returned value in a later calculation. Try the recall before opening notes.

Your learning target

I can use a returned value in a later calculation.

I can trace **local variable**s separately for each function call.

I can detect a missing return and test repeated calls.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Functions and arguments. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Expressions and operator order. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

This week you will practise:

Use a returned value in a later calculation

Trace **local variable**s separately for each function call

Detect a missing return and test repeated calls

Retrieve what you already know

> A function show_remaining(capacity, used) prints capacity - used. What does calling it with (9, 4) display? Does defining it alone display that result?

My starting explanation

This week's end goal

Trace **return value**s and local state and explain the changed program before running it.

Notice the evidence

> Investigate the supplied Return values and local state case. Separate observations, given facts and assumptions before choosing an explanation.

Two fictional cupboards have spare spaces. We want to add their spare spaces without copying numbers from the screen. Last week a function printed each answer.

This week a function returns its answer so the calling code can store it, combine it or choose to print it.

Assume nonnegative integer counts and used no greater than capacity. This calculation does not validate those assumptions.

Work in a Python script so a call does not automatically display its **return value** as it might in an interactive console.

> **List the given facts and one fact that is not supplied.**

STUDENT LESSON · 3 OF 18

See how it works

> Follow the mechanism for Return values and local state. Connect each labelled input, deciding step and result.

```
def remaining(capacity, used):
    spaces = capacity - used
    return spaces

first = remaining(12, 4)
```

```
second = remaining(10, 7)
print(first + second)
```

Two calls return two values

CALL	INSIDE THIS CALL	BACK IN THE CALLER
remaining(12, 4)	spaces becomes 8	first receives 8
remaining(10, 7)	spaces becomes 3	second receives 3

The caller adds the returned values. Only print(first + second) displays 11.

return spaces ends the current function call and sends the value of spaces back. The assignment first = remaining(12, 4) waits for that value and stores 8. The next call stores 3 in second. Neither call prints.

The final statement prints 11. A statement after an executed return in the same call is not reached.

Connect the picture and the explanation

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

STUDENT LESSON · 4 OF 18

Compare the cases

> Compare the supplied cases for Return values and local state. Identify the detail that changes the conclusion.

Compare a familiar case with a changed case

Using the correct definition, store a = remaining(8, 3), b = remaining(8, 8), c = remaining(8, 3). State a, b and c. In a fresh script, explain why print(spaces) outside the function fails.

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change one thing

Change: Add print(result) after the calls in a fresh script with no global result.

Predicted result

Prediction: NameError occurs because that local name is not defined in the **caller**.

Why the result follows

Reason: A returned value and the function's internal name are different things.

STUDENT LESSON · 5 OF 18

Words that unlock the idea

Use the vocabulary of Return values and local state to explain a concrete decision, rather than only memorising definitions.

Word	Meaning
Return value	A value a function sends back to its caller
Caller	The code that calls a function
Local variable	A name assigned inside a function and used in that function call
Scope	The region of code in which a name can be looked up
None	A Python value representing no result value in this lesson

Use two terms to explain a step in this week's example.

STUDENT LESSON · 6 OF 18

Follow a worked example

Follow the worked trace for Return values and local state, predict a step, then compare with its explanation.

Point in execution	Local spaces	Caller values
First call before return	8	first not assigned yet
Second call before return	3	first is still 8
After both calls	Not accessible by this local name	first is 8 and second is 3

Worked explanation · Return values and local state

capacity, used and spaces belong to the current call. The second call gets new parameter values and calculates its own spaces; it does not overwrite first.

In a fresh script with no outside variable called spaces, adding print(spaces) after the calls raises NameError. Use the returned value stored in first or second instead.

Change the calls to remaining(9, 9) and remaining(9, 0). What values are stored and what does their sum print? What happens if only the final print statement is removed?

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Return values and local state

Return values survive their calls. A function returns a local calculation that the **caller** stores.

The facts we will use

Supplied facts

Python 3: def extra(a,b): result=a+b; return result. Then first=extra(2,5); second=extra(4,1); print(first,second).

AI ACADEMY / PYTHON-BRIDGE/9

Return values and local state

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.

1. Trace first call

Its local result is 7; return assigns 7 to first.

2. Trace second call

Its new local result is 5; return assigns 5 to second.

3. Read caller values

The final print displays 7 5; the second call does not overwrite first.

CHANGE ONE THING

Add print(result) after the calls in a fresh script with no global result.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Trace first call	Its local result is 7; return assigns 7 to first.
2. Trace second call	Its new local result is 5; return assigns 5 to second.
3. Read caller values	The final print displays 7 5; the second call does not overwrite first.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Why the result follows

Returning passes a value back to the **caller**. Local names belong to a function call; storing the returned value creates a separate **caller** binding.

What this example does not establish

Keep the stated limit

The example uses immutable integers; shared mutable objects require additional reasoning about references.

Keep the stated limit

Read the labelled picture: Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

Keep the stated limit

Change one thing: Add print(result) after the calls in a fresh script with no global result. Predict the result before checking the explanation. What remains the same?

STUDENT LESSON · 7 OF 18

Find and repair a mistake

> Find the unsupported step in the Return values and local state claim and explain a repair using the supplied evidence.

```
def remaining(capacity, used):
    spaces = capacity - used
    print(spaces)

answer = remaining(12, 4)
print(answer)
```

This faulty version displays 8 and then **None**. It reaches the end of its body without a return statement, so the call returns **None**. Printing 8 inside the function does not make 8 its **return value**. **None** is not zero.

Trying answer + 1 raises TypeError in this example.

Replace print(spaces) inside the function with return spaces. Keep print(answer) in the **caller**. The repaired script prints 8 once, and the **caller** can calculate answer + 1.

Test the stored value as well as what appears on screen.

Locate the earliest unsupported step

Use the faulty printing version with answer = remaining(11, 2). State the displayed number and the value stored in answer. Explain why print(answer + 1) fails, then repair the function and predict the repaired **output**.

> **State one repair and the checks you would rerun.**

STUDENT LESSON · 8 OF 18

Practise together

> Practise Return values and local state with support, then explain the deciding step in your own words.

Using the correct worked definition, execute result = remaining(15, 6), then doubled = result * 2. State both stored values and what the script displays before a print statement is added.

> **Change the calls to remaining(9, 9) and remaining(9, 0). What values are stored and what does their sum print? What happens if only the final print statement is removed?**

Explain your prediction to a partner before comparing results.

Practise with less help: Trace separate calls

Use workbook W2 for the connected example "**Return value**s and local state". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for **Return value**s and local state. Proposed explanation: "Printing a value returns that value". Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

STUDENT LESSON · 9 OF 18

Run the investigation

> Investigate Return values and local state. Record the prediction, actual observation or labelled paper trace, explanation and next test separately.

Predict, investigate and preserve the evidence

Using the correct definition, store a = remaining(8, 3), b = remaining(8, 8), c = remaining(8, 3). State a, b and c. In a fresh script, explain why print(spaces) outside the function fails.

```
def remaining(capacity, used):
    spaces = capacity - used
    return spaces

first = remaining(12, 4)
second = remaining(10, 7)
print(first + second)
```

```
def remaining(capacity, used):
    spaces = capacity - used
    print(spaces)

answer = remaining(12, 4)
print(answer)
```

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

STUDENT LESSON · 10 OF 18

Build your understanding

> Move from recall to tracing, diagnosis and transfer in Return values and local state. Explain why each step is justified.

Explain → apply → challenge

Using the correct worked definition, execute result = remaining(15, 6), then doubled = result * 2. State both stored values and what the script displays before a print statement is added.

Use the faulty printing version with answer = remaining(11, 2). State the displayed number and the value stored in answer. Explain why print(answer + 1) fails, then repair the function and predict the repaired **output**.

A function body has spaces = capacity - used, then return spaces, then print("done"), all at the same indentation. What does answer = remaining(4, 4) store and display? Explain whether the final print can run.

> **Explain which evidence supports your conclusion and which claim remains unproved.**

STUDENT LESSON · 11 OF 18

Connect your project

> Apply this week's idea to Python readiness portfolio. Keep an annotated decision and the evidence supporting it.

Add a labelled **return value**s and local state **evidence** sheet to Python readiness portfolio. Show how you can use a returned value in a later calculation.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

STUDENT LESSON · 12 OF 18

Show what you know

> Use a genuinely new case for independent evidence. Previously discussed examples remain practice; use the separately named independent case before feedback.

For a function that only calculates from these integer arguments, repeat the first call after a different call. remaining(12, 4), remaining(10, 7), remaining(12, 4) should return 8, 3, 8.

Store each result and compare it with a **prediction**. These examples create fresh **local variable**s each time; do not assume every possible function is independent of earlier calls.

> **A learner says that a function is correct because it prints 8. Explain how you would check that its caller actually receives 8. Include a second call and say why two passing tests are limited evidence.**

A function body has spaces = capacity - used, then return spaces, then print("done"), all at the same indentation. What does answer = remaining(4, 4) store and display? Explain whether the final print can run.

My limited claim and a useful next check

STUDENT LESSON · 13 OF 18

Study a complete case

> Revisit the complete Return values and local state case for supported practice. Repeated examples do not become fresh assessments.

Complete evidence record

Two calls return two values

CALL	INSIDE THIS CALL	BACK IN THE CALLER
remaining(12, 4)	spaces becomes 8	first receives 8
remaining(10, 7)	spaces becomes 3	second receives 3

The caller adds the returned values. Only print(first + second) displays 11.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

STUDENT LESSON · 14 OF 18

Try a new case independently

> Attempt the reserved new case before feedback. Record support; the earlier examples below remain additional practice.

Independent case: Use returns and keep local state local

> Trace both calls and the final expression. If return total is removed from the function, does this final print complete? Would a separate print(total) outside the function have access to its **local variable**?

```
def twice(n):
    total = 2 * n
    return total
first = twice(3)
second = twice(5)
print(first, second, first + second)
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Each tray holds 5 counters. Define stock(trays, loose) to return the total using a **local variable** total. Assume nonnegative integer arguments. Store results for (2, 3), (0, 3) and (2, 0).

Print the sum of the first two stored results. Predict that **output** and all three stored values. Call (2, 3) again to check fresh state.

> **My independent procedure, result and limitation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Record support used: **none**, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

Your lesson route

> Use this map to navigate Return values and local state, connect the goals to tasks and keep one evidence record for each attempt.

Start with this question: **Return value**s survive their calls. A function returns a local calculation that the **caller** stores.

By the end, I can	How I will show it
Use a returned value in a later calculation	Explain the deciding step, show a trace or test, and state one limit.
Trace local variables separately for each function call	Explain the deciding step, show a trace or test, and state one limit.
Detect a missing return and test repeated calls	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

STUDENT LESSON · 16 OF 18

Finish and return

> Explain Return values and local state without notes, identify a remaining question and plan a later check with a changed case.

Close your notes. Explain this goal in your own words: Use a returned value in a later calculation

My deciding step, evidence and one remaining question

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

STUDENT LESSON · 17 OF 18

Supported practice, repair and portfolio

> Attempt supported questions before using hints or feedback. Review a missing building block, then keep the completed work.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Return values survive their calls. A function returns a local calculation that the **caller** stores.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? The final print displays 7 5; the second call does not overwrite first.

1. Read **caller** values

2. Trace first call

3. Trace second call

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Feedback · compare after trying

After your attempt, compare with the supported explanation: Read **caller** values The final print displays 7 5; the second call does not overwrite first. Returning passes a value back to the **caller**.

Local names belong to a function call; storing the returned value creates a separate **caller** binding.

Practice 2: explain the deciding evidence

Changed case · predict first

Changed-case practice: Add print(result) after the calls in a fresh script with no global result. Which conclusion is supported by the supplied facts?

1. We can decide the result without examining the changed **input** or the procedure.

2. NameError occurs because that local name is not defined in the **caller**.

3. This one example guarantees correct results for every future **input**.

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Feedback · compare after trying

After your attempt, compare with the supported explanation: NameError occurs because that local name is not defined in the **caller**. A returned value and the function's internal name are different things.

Repair a missing building block

Review Functions and arguments (python-bridge/8). Goal: Distinguish a function definition from a function call. Short practice: Match every argument to a parameter for the two calls.

Why do **output**s differ, and what prints when both calls are removed?. Return to this week and explain what you changed.

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: List the four visited values and their running totals.

Explain why 5 is absent and what prints for range(1,1).. Return to this week and explain what you changed.

Review Expressions and operator order (python-bridge/3). Goal: Evaluate numeric expressions using Python’s grouping rules. Short practice: Trace the operation order in both expressions.

Identify which picture step changes with parentheses and predict both results when the multiplier becomes four.. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

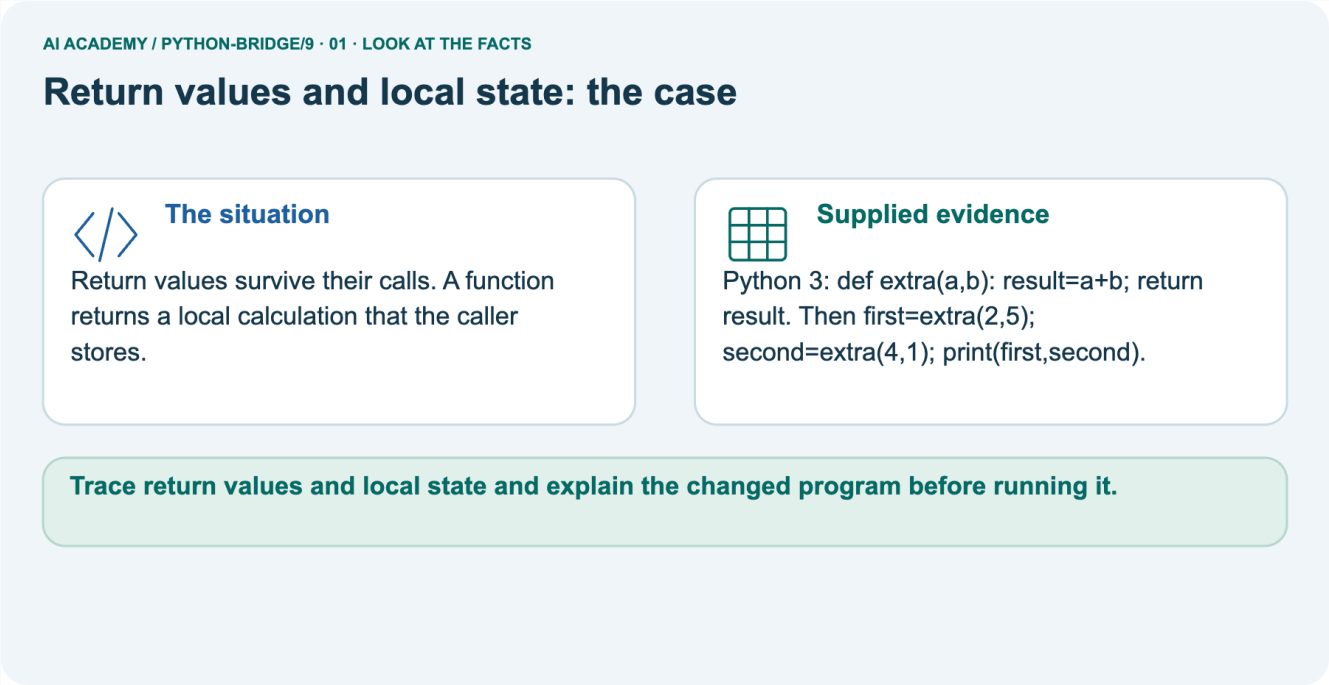
Drafts stay in this browser when storage is available; clear drafts on a shared device.

STUDENT LESSON · 18 OF 18

Visual concept journey

> Explain each Return values and local state graphic using its labels, deciding evidence and relevant limitation.

01. Return values and local state: the case



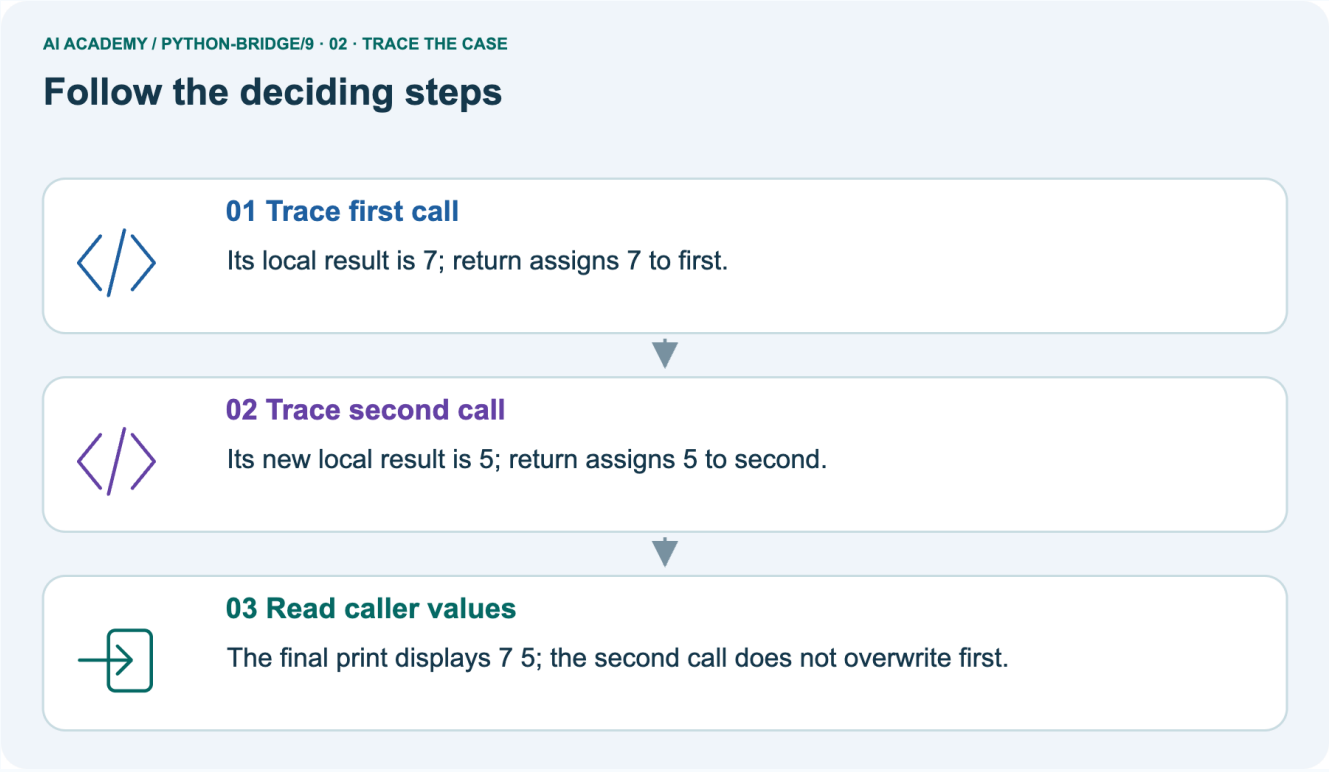
Follow the labelled picture with the matching explanation. Open the vector diagram

Return values survive their calls. A function returns a local calculation that the **caller** stores. Supplied facts: Python 3: def extra(a,b): result=a+b; return result.

Then first=extra(2,5); second=extra(4,1); print(first,second).

> Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Trace first call: Its local result is 7; return assigns 7 to first. → Trace second call: Its new local result is 5; return assigns 5 to second. → Read **caller** values: The final print displays 7 5; the second call does not overwrite first.

> Point to each numbered stage and explain which recorded **input** it uses.

03. Two calls return two values

AI ACADEMY / PYTHON-BRIDGE/9 · 03 · INSPECT THE LESSON MODEL

Two calls return two values

Call	Inside this call	Back in the caller
remaining(12, 4)	spaces becomes 8	first receives 8
remaining(10, 7)	spaces becomes 3	second receives 3

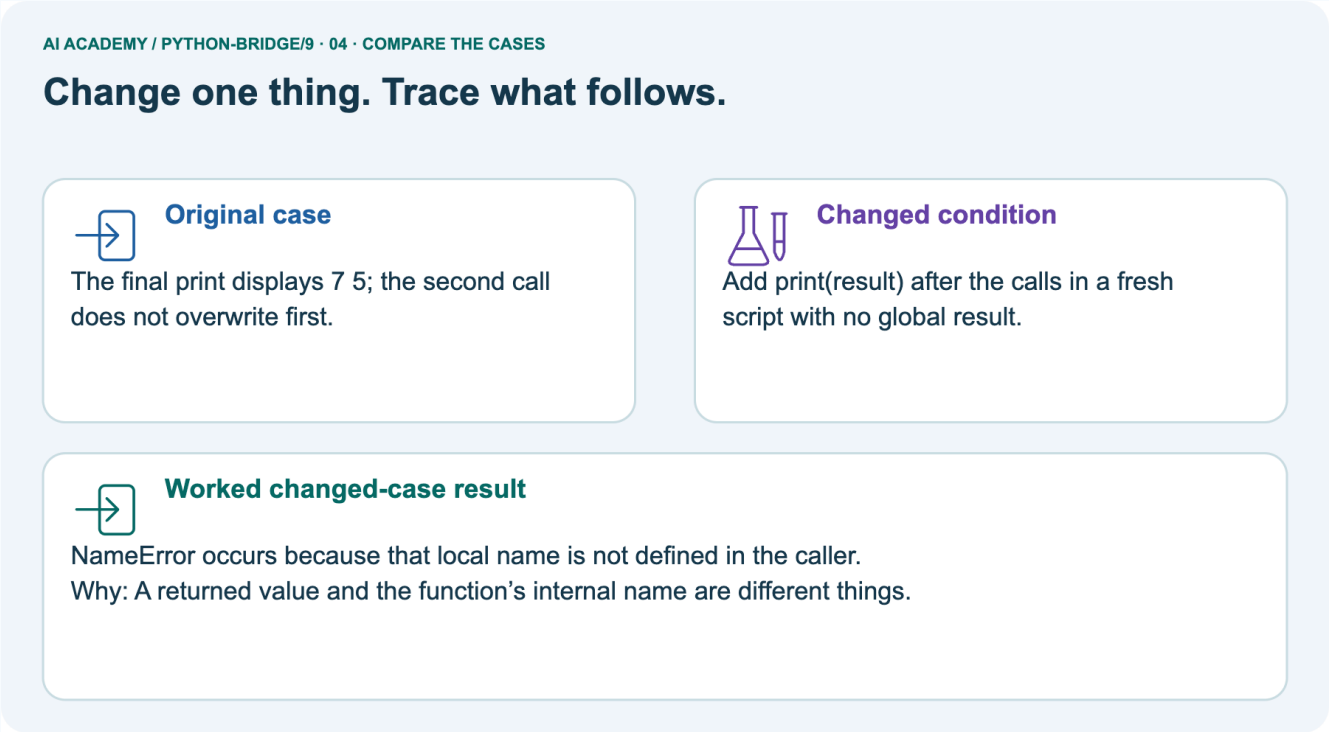
The caller adds the returned values. Only print(first + second) displays 11.

Follow the labelled picture with the matching explanation. Open the vector diagram

Two calls return two values. remaining(12, 4) | spaces becomes 8 | first receives 8. remaining(10, 7) | spaces becomes 3 | second receives 3

> Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



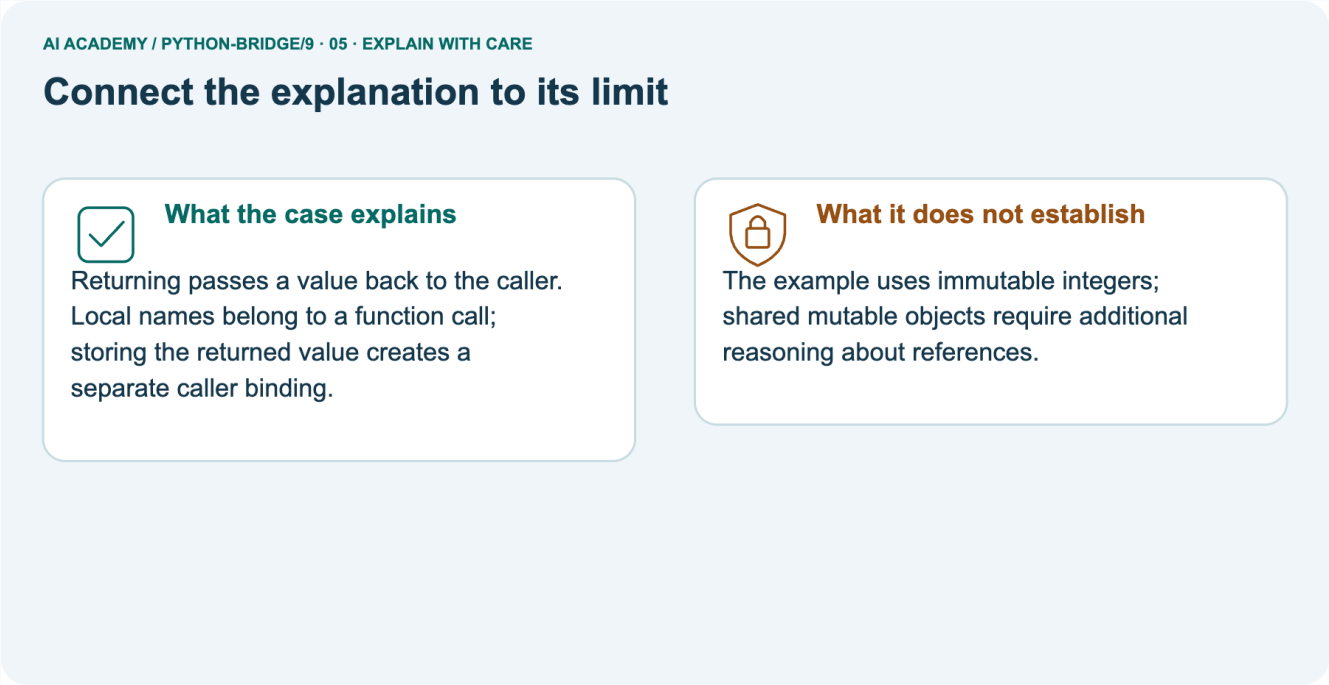
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: The final print displays 7 5; the second call does not overwrite first. Changed condition: Add print(result) after the calls in a fresh script with no global result.

Expected result: NameError occurs because that local name is not defined in the **caller**. Reason: A returned value and the function’s internal name are different things.

> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



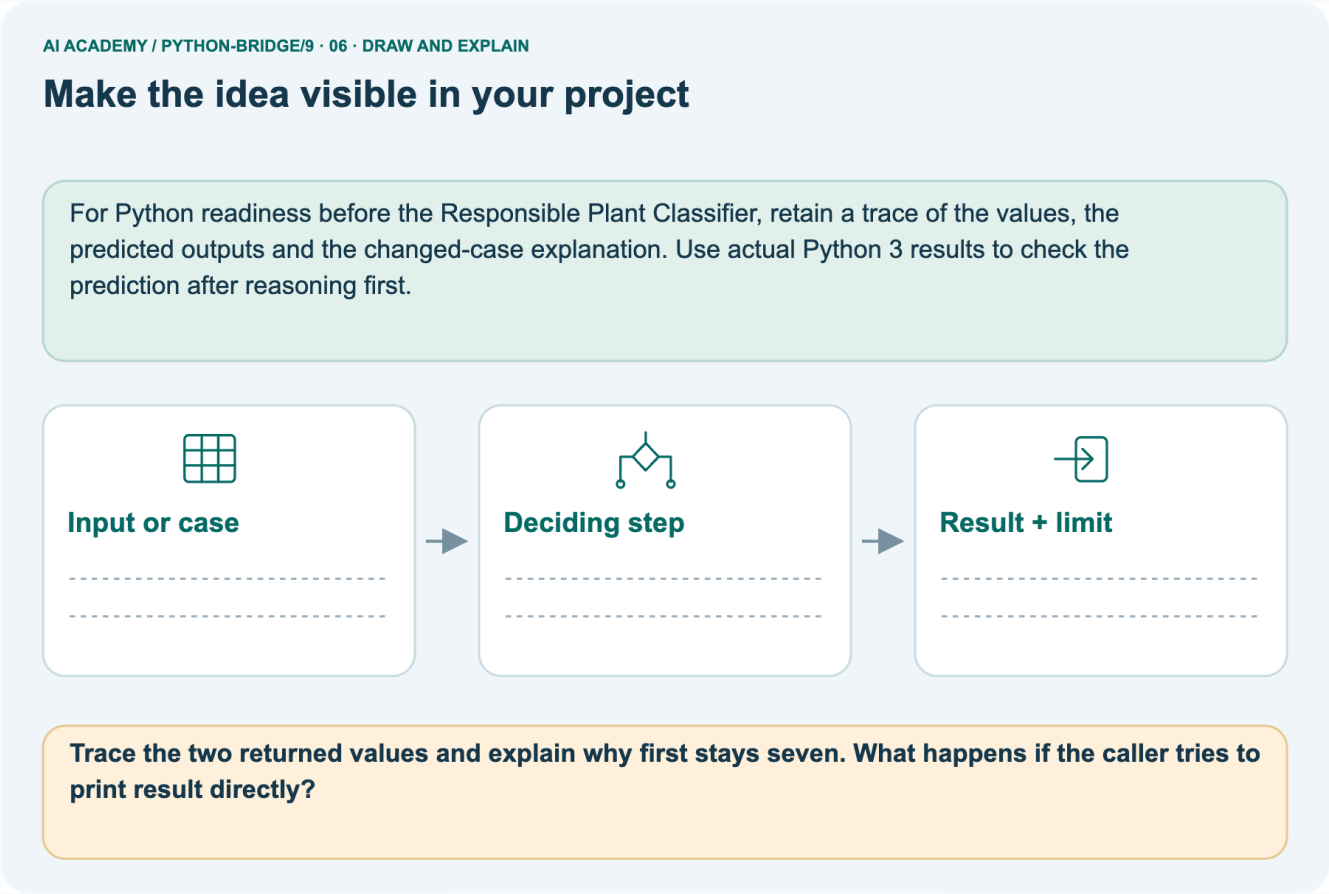
Follow the labelled picture with the matching explanation. Open the vector diagram

Returning passes a value back to the **caller**. Local names belong to a function call; storing the returned value creates a separate **caller** binding.

Limit: The example uses immutable integers; shared mutable objects require additional reasoning about references.

> Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

> Draw your project trace and justify the deciding step using **evidence**.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.